

# Chiseling101

July 30, 2026

## 1 Chiseling 101

CC-BY 2026 Brooksbank, Kassabov, Wilson

Dleto (Chisel) detects and locates sparsity patterns in tensor data. This notebook demonstrates the basic functionality of Dleto, how to select different chisels, and what to expect from the outputs.

1. Tucker Decompositions
2. Block Decompositions
3. Stratification

### 1.1 1. Tucker Decompositions

We first use Dleto to find familiar tensor structures known as Tucker Decompositions. You can think of these as tensor generalizations of matrix nullspaces.

**Performance.** Tucker decompositions have been explored since the 1800's, and there are highly optimized strategies to discover them. This tutorial uses a familiar problem to explore the Dleto's functionality, but the performance of our demonstration is far from optimal on this particular application. Due to its generality, Dleto chiseling operates with a complexity slightly greater than more direct Tucker Decomposition strategies. Indeed Dleto itself leverages such strategies through its function `nondeg`.

A **Tucker Decomposition** of a tensor  $\Gamma$  framed by *axes* (known also as *modes*, *legs*, or *indices*)  $\mathbb{K}^{d_1}, \dots, \mathbb{K}^{d_\ell}$ , is a subset  $A \subset \{1, \dots, \ell\}$  and a decomposition

$$\forall a \in A, \quad \mathbb{K}^{d_a} = E_a \oplus R_a$$

such that the contraction of  $\Gamma$  on  $R_a$  is 0.

The decompositions can be given by partitioned bases of  $E_a$  and  $R_a$ , or equivalently by linear projections  $e_a : \mathbb{K}^{d_a} \rightarrow \mathbb{K}^{e_a}$  with kernel  $R_a$ . In many situations the purpose of a Tucker decomposition is to restrict the tensor to the  $E_a$ -spaces, which we call the “truncated” form.

#### 1.1.1 1.1 Loading Dleto

Start by loading `Dleto.jl`. First time users may need to install auxiliary packages and possibly set up Julia for notebooks. That is a one-time setup for which we provide instructions. Alternatively, you can use the links on the landing page to access the online Binder demonstration.

### 1.1.2 !Colab users begin here!

While [Google's Colab](#) supports Julia, it's default configuration is `Python` and you will need to invite Colab to use Julia before proceeding, which can be done either by selecting the Runtime or by running the following cell first.

```
[1]: # %% julia
using Pkg
Pkg.add( "IJulia" )
Pkg.add( "ITensors" )
Pkg.add( "Plots" )

Resolving package versions...
Compat entries added for
Project No packages added to or removed from
`~/CODE/OpenDleto/Project.toml`
Manifest No packages added to or removed from
`~/CODE/OpenDleto/Manifest.toml`
Resolving package versions...
Compat entries added for
Project No packages added to or removed from
`~/CODE/OpenDleto/Project.toml`
Manifest No packages added to or removed from
`~/CODE/OpenDleto/Manifest.toml`
Resolving package versions...
Compat entries added for
Project No packages added to or removed from
`~/CODE/OpenDleto/Project.toml`
Manifest No packages added to or removed from
`~/CODE/OpenDleto/Manifest.toml`
```

### 1.1.3 All other users begin here.

If you are running from your own Jupyter environment, make sure to have installed a Julia kernel (version 1.11 or higher) and configured Jupyter to use Julia.

```
[2]: # Uncomment and run the first time, if Dleto is not installed
using Pkg

# Option 1: To install from remote repository, use:
# Pkg.add(url="https://github.com/thetensor-space/OpenDleto")

# Option 2: If cloned locally at PATH
Pkg.activate( "../" )
Pkg.instantiate()

Activating project at `~/CODE/OpenDleto`
```

Once you have added Dleto to your Julia packages you can load all of the packages we will need: `ITensors` for general tensor controls; `Plots` for visualization tools; and of course `Dleto` itself, the

primary package of chisel techniques.

```
[3]: using ITensors
      using Pkg; Pkg.add( "PlotlyJS" ); using PlotlyJS # Uncomment if you don't have
      ↪PlotlyJS installed
      using Plots: plot
      using Dleto
      using Dleto:      # Explicitly import from Dleto to resolve ambiguity
```

```
Resolving package versions...
Compat entries added for
Project No packages added to or removed from
`~/CODE/OpenDleto/Project.toml`
Manifest No packages added to or removed from
`~/CODE/OpenDleto/Manifest.toml`
```

```
WebIO._IJuliaInit()
```

```
Loading Dleto Plots Extension
```

Select your viewer properties, `:widescreen` for laptop & desktop viewing and `:vertical` for vertical screens like tablets/phones:

```
[4]: layout=(1,2); pic_size=(900, 400); set_compare_layout(:widescreen)
      # layout=(2,1); pic_size=(400, 900); set_compare_layout(:vertical)
```

#### 1.1.4 1.2 Creating a Tensor Experiment

We create two tensors, one randomized for control, and one identical in size but with 2 rows, 2 columns, and 2 slices set to approximately zero. We then randomize the experiment tensor by applying a change in coordinates. We will demonstrate how to use chiseling to detect and recover the hidden zero rows / columns / slices.

**Notation.** Julia accepts L<sup>A</sup>T<sub>E</sub>X styled commands with tab-completion. For example, to insert the Unicode character for  $\Gamma$  use `\Gamma` in the code area followed by `tab` (or cut-and-paste a character you see somewhere else). If you prefer, you can replace these characters with strings of characters of your choosing. `Dleto.jl` calculations make substantial use of tensors, matrices, and lists of matrices, so we recommend using a convention that clearly delineates roles.

We will use: \* Greek capitals such as  $\Gamma$  (`\Gamma`),  $\Delta$  (`\Delta`),  $\Sigma$  (`\Sigma`),  $\Xi$  (`\Xi`),  $\Upsilon$  (`\Upsilon`) for tensors; \* English capitals such as  $X$ ,  $Y$ ,  $Z$  for matrices; \* Greek lower-case letters for real numbers; \* English lower-case letters for integers; and \* variables that suggest plurality, such as  $\Gamma$ s and  $X$ s, for lists.

We recommend using the existing parameters in your first pass through the notebook. At the end of the section we encourage you to revisit cells and change the parameters to observe the changes.

```
[5]: tol = 1e-6          # How sensitive the chiseling is

      # Creates a control tensor
```

```

ds = (7,6,5)          # Increase to (25,25,25)
Γ = randn(Float64, ds)

as=(4,4,3)
Δ = randn(Float64, as)  zeros(Float64, ds .- as)

# Add noise to experiment tensor
# db = 0.001; tol = 10*db          # Add some noise, increase tolerance
# Δ += db*randn(Float64, ds)

compare(Γ, Δ; left_title="Control Γ", right_title="Experiment Δ")

```

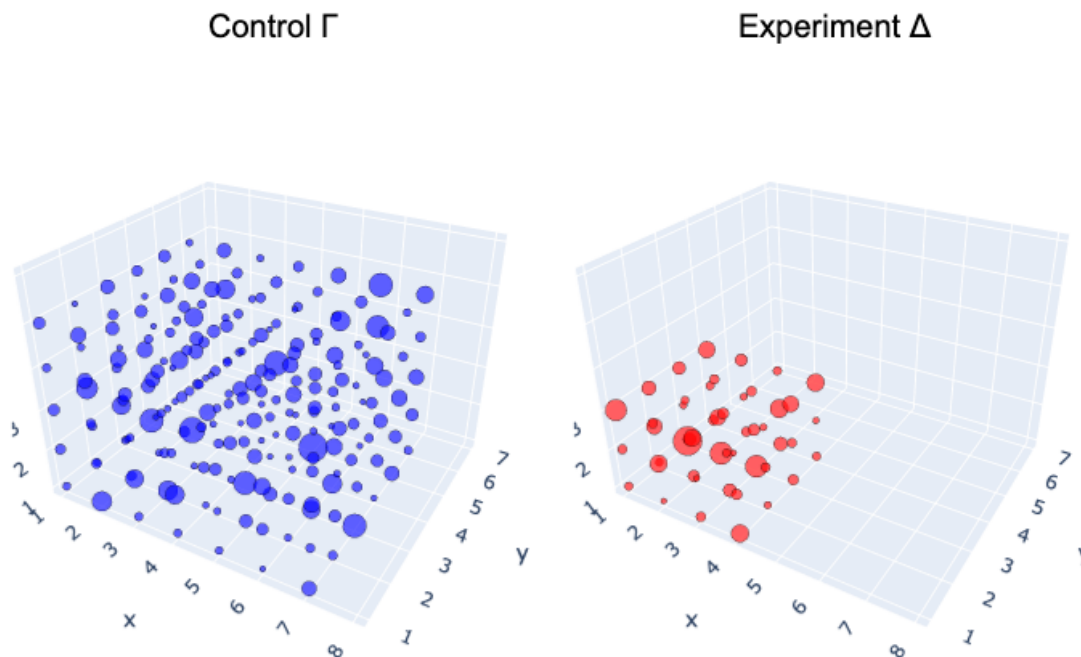
Visualization tools are very useful for larger tensors:

```

[6]: p1 = plot_tensor(Γ; title="Control Γ", color=:blue)    # tol bounds how much
      ↪ noise is shown
p2 = plot_tensor(Δ; title="Experiment Δ", color=:red)
plot(p1, p2, layout=layout, size=pic_size) # Side-by-side plots

```

[6]:



The control tensor is unlikely to have large regions of zeros, while the experiment tensor is constructed so that it does. You will observe changes when you adjust the parameters.

### 1.1.5 1.3 Tensor Randomization

Dleto detects features such as redundancy without knowing about them in advance. We provide a demonstration. First, we scramble the experiment tensor to obscure its features. The scrambling process applies random basis changes to each of the original coordinate axes. For convenience, Dleto has a few built in randomization procedures, but you can create and apply your own list of matrices if you wish. For instance, you can replace `X1s=[ A[1], A[2], A[3]]` where each `A[i]` is an `ITensor` matrix with dimensions matching `ds` and define  `$\Gamma_{\text{rand}} = \Gamma * X1s$` .

Dleto uses `ITensors.jl` for most of its tensor management, and it provides methods to convert standard Julia arrays to `ITensors`. You can use `Array(data, inds(data))` to convert back to arrays, but `ITensors` has a number of convenient features for tensors, and it organizes information to avoid common errors. We therefore recommend using `ITensors`.

```
[7]:  $\Gamma_{\text{rand}}, X1s = \text{randomize\_tensor}(\Gamma)$ 
      @assert isapprox( $\Gamma * X1s$ ,  $\Gamma_{\text{rand}}$ )

       $\Delta_{\text{rand}}, Y1s = \text{randomize\_tensor}(\Delta)$ 
      @assert isapprox( $\Delta * Y1s$ ,  $\Delta_{\text{rand}}$ )
```

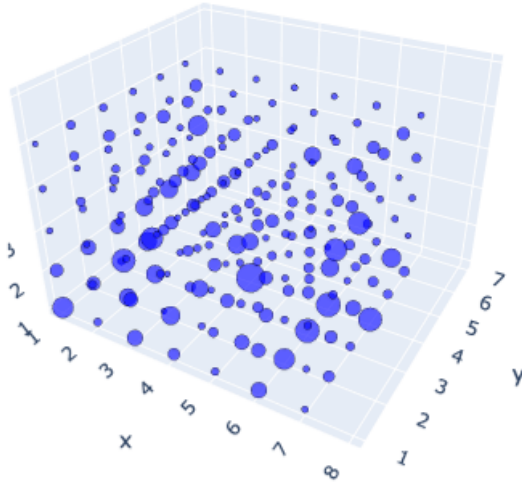
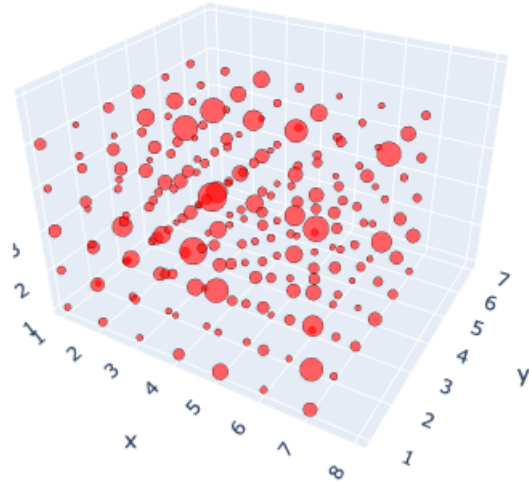
The control and the experiment tensor now look similarly unstructured:

```
[8]: compare( $\Gamma_{\text{rand}}$ ,  $\Delta_{\text{rand}}$ ; left_title="Control  $\Gamma$ ", right_title="Experiment  $\Delta_{\text{rand}}$ ")
```

Perhaps this is easier to see with a plot:

```
[9]: p1 = plot_tensor( $\Gamma_{\text{rand}}$ , tol; title="Scrambled Control  $\Gamma$ ", color=:blue)
      p2 = plot_tensor( $\Delta_{\text{rand}}$ , tol; title="Scrambled Experiment  $\Delta$ ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

```
[9]:
```

Scrambled Control  $\Gamma$ Scrambled Experiment  $\Delta$ 

### 1.1.6 1.4 Chiseling for Tucker Decompositions

We now use Dleto to find a Tucker decomposition. The key command is `stratify`. While this function can be used with multiple parameters, it has default settings that detect structure when it is present and often perform well as a first approximation.

```
[10]: # Γ_strat, X2s = stratify(Γ_rand)
Γ_strat, X2s = stratify(Γ_rand; tol=tol)    # specify tol to match noise level
@assert isapprox(Γ_rand*X2s, Γ_strat)
# Δ_strat, Y2s = stratify(Δ_rand)
Δ_strat, Y2s = stratify(Δ_rand; tol=tol)
@assert isapprox(Δ_rand*Y2s, Δ_strat)
```

[ Info: Found 2 derivations for stratification.

[ Info: Found 10 derivations for stratification.

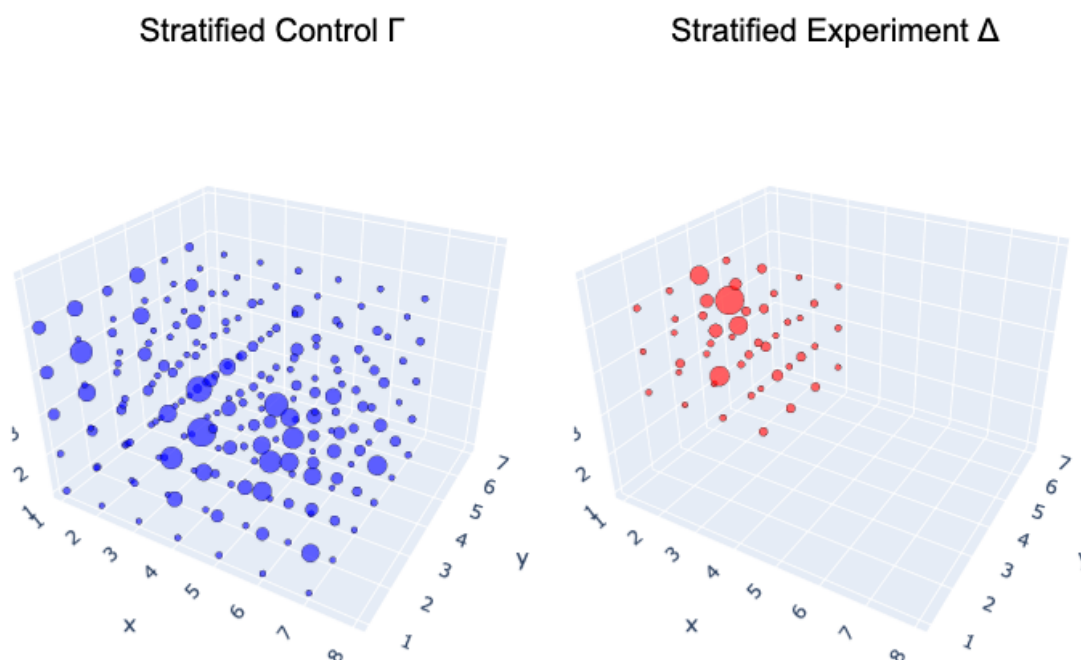
The output is a tensor-transform pair. You may observe some extra information printed that reports the total number “derivations” detected during stratification. Random dense tensors typically admit only 2 derivations. When a Tucker decomposition exists in a tensor, on the other hand, you will typically see more than 2 derivations. Indeed you can think of a derivation number greater than 2 as a proxy for Dleto having detected something of interest in your tensor.

**Round 2?** If you modified the parameters you may observe that stratification no longer distinguishes the experiment tensor. You now need to consider how much tolerance to pass along to the stratification by adding the optional parameter `tol=tol` (the tolerance we set above). Replace the original commands by the commented commands to include this tolerance adjustment. You will know that your tolerance is adequate if the number derivations for the experiment is more than 2.

Let us look at the results:

```
[11]: p1 = plot_tensor( $\Gamma$ _strat, tol; title="Stratified Control  $\Gamma$ ", color=:blue)
      p2 = plot_tensor( $\Delta$ _strat, tol; title="Stratified Experiment  $\Delta$ ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

[11]:



If the noise-to-dimension ratio is reasonable, you should observe that the experiment tensor now clusters nearly all its values in one region. Although the location of that region may shift from experiment to experiment, it should have similar volume to the initial nonzero region in the experiment tensor.

This is a good place to return to the start of this section and begin altering the parameters to see how the results change.

\* First you could increase the dimension. **Caution:** the complexity of a tensor is proportional to the volume. Changing (5,5,5) to (25,25,25) increases the input by a factor of 125, not 5! You should therefore proceed with caution until you acclimate to the impact your changes have on performance

and memory. \* Secondly, you could increase the background noise and set the tolerance to match. For example, `tol=2*db` is an appropriate range.

### 1.1.7 1.5 Optimal Tucker Decompositions

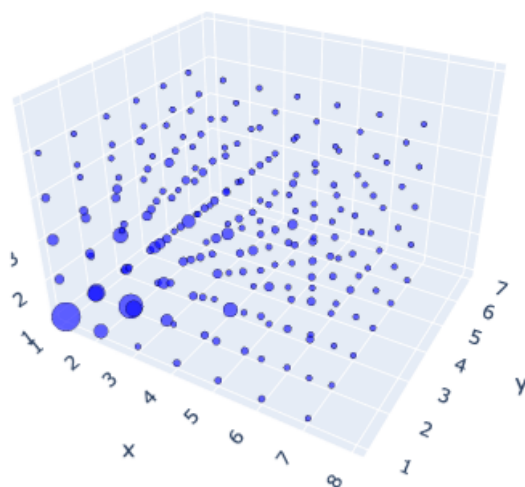
As noted earlier, Dleto has a function `nondeg` to perform Tucker computations more efficiently. The function accepts two modes `:full` to regroup the nonzero values inside the original space, and `:trunc` to truncate the detected zero rows, columns, slices. It is usually advisable to pass to a nondegenerate tensor using `nondeg` at the start of any computation because it has a relatively low cost and the dimension of the resulting tensor can be (sometimes quite substantially) smaller. This can reduce the cost of all subsequent computations.

```
[12]: Γ_nondeg, X3s = Dleto.nondeg(Γ_rand, mode=:full); # use mode=:trunc to
      ↪ truncate the 0's in the Tucker decomposition
@assert isapprox(Γ_rand * X3s, Γ_nondeg)
Δ_nondeg, Y3s = Dleto.nondeg(Δ_rand, mode=:full);
@assert isapprox(Δ_rand * Y3s, Δ_nondeg)

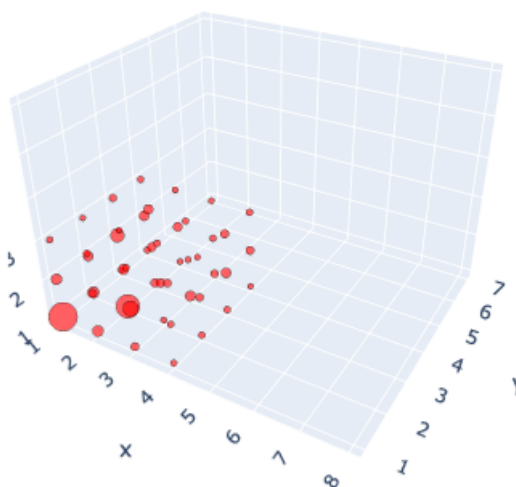
p1 = plot_tensor(Γ_nondeg, tol; title="Tucker Control Γ", color=:blue)
p2 = plot_tensor(Δ_nondeg, tol; title="Tucker Experiment Δ", color=:red)
plot(p1, p2, layout=layout, size=pic_size)
```

[12]:

Tucker Control  $\Gamma$



Tucker Experiment  $\Delta$



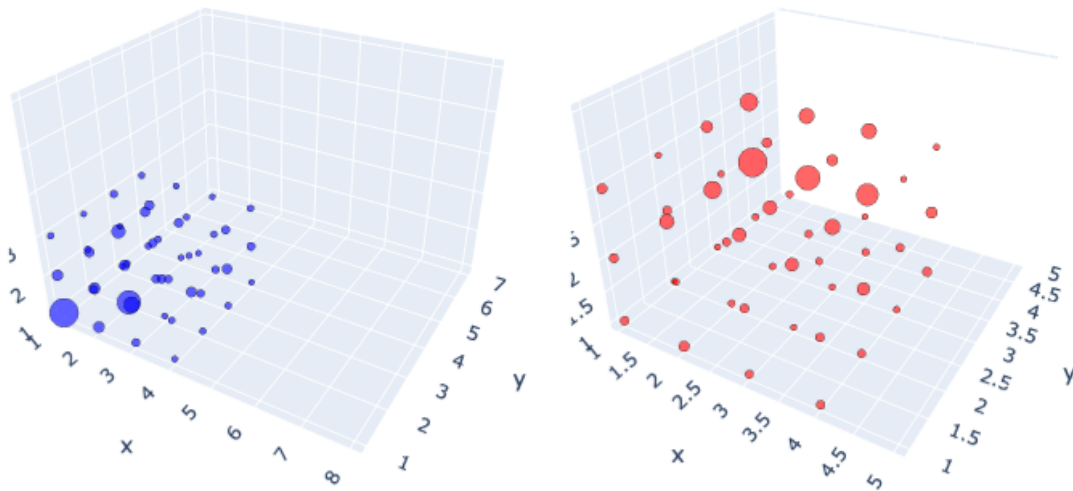
```
[13]: Δ_nondeg_trunc, Y4s = Dleto.nondeg(Δ_rand, mode=:trunc);
@assert isapprox(Δ_rand * Y4s, Δ_nondeg_trunc)

p1 = plot_tensor(Δ_nondeg, tol; title="Tucker Experiment Δ full", color=:blue)
p2 = plot_tensor(Δ_nondeg_trunc, tol; title="Tucker Experiment Δ truncated",
    ↪color=:red)
plot(p1, p2, layout=layout, size=pic_size)
```

[13]:

Tucker Experiment  $\Delta$  full

Tucker Experiment  $\Delta$  truncated



Tucker decompositions are closures: repeated applications achieve no further gains (though it may reorder and rescale the one it has already carried out):

```
[14]: Δ_nondeg_trunc, _ = nondeg(Δ_rand, mode=:trunc);
Δ_nondeg2_trunc, _ = nondeg(Δ_nondeg_trunc, mode=:trunc);
@assert size(Δ_nondeg_trunc) == size(Δ_nondeg2_trunc)
```

If you did not already change parameters above, we invite you to return to the first cell and do so now. We can continue to other experiments when you are ready!

### 1.1.8 Carry out the trash

While it is helpful to visualize the effects of Dleto chiseling, these 3D plots (especially of large dense randomized data) can use up much the available memory of a notebook environment. If you have

limited resources to run the notebook or experience lag times then you may benefit from running the following garbage collection routines between sections. If you are very tight on memory budgets you may wish to clear output data of the 3D plots above.

```
[15]: # release memory of tensors
Γ =nothing; Δ=nothing
Γ_rand=nothing; Δ_rand=nothing
Γ_strat=nothing; Δ_strat=nothing
X1s=nothing; Y1s=nothing
X2s=nothing; Y2s=nothing

# release memory plots. Note, in some Jupyter environments, the 3D plots will
↳ persist as an image but no longer be interactive.
p1=nothing
p2=nothing
GC.gc()
```

## 2. Block Diagonalization

You can think of performing a Tucker decomposition as finding a single block on the diagonal of a tensor. It is of course natural to ask whether one can find several distinct blocks (assuming they exist). In similar fashion to the first experiment, we now create such a tensor and hide the blocks to see if Dleto chiseling can recover them.

```
[16]: as=[3,2,7]
bs=[4,5,2]
cs=[5,3,3]
ds=as+bs+cs

Γ = randn(ds...); # a control tensor

Δ = randn(as...) randn(bs...) randn(cs...)

# Add noise to experiment tensor
# db = 0.001; tol = 10*db # Add some noise, increase tolerance
# Δ += db*randn(Float64, ds)

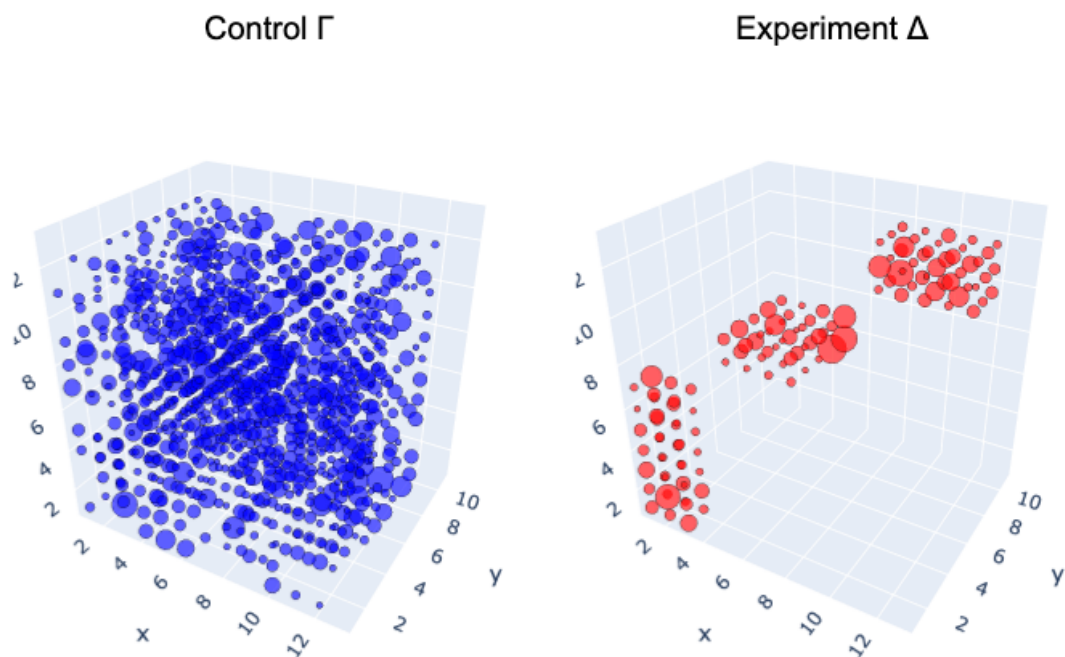
p1 = plot_tensor(Γ; title="Control Γ", color=:blue)
p2 = plot_tensor(Δ; title="Experiment Δ", color=:red)
plot(p1, p2, layout=layout, size=pic_size)
```

**Warning:** More than 1,000 viewable entries, plotting only the largest points.

@ DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257

[16]:



We again hide the structure by randomizing the reference frame of our tensor:

```
[17]: Γ_rand, X1s = randomize_tensor(Γ);
      Δ_rand, Y1s = randomize_tensor(Δ); # In theory Δ_rand = Δ*Xs

      # Plotted these two tensors are essentially indistinguishable from each other.
      p1 = plot_tensor(Γ_rand; title="Scrambled Control Γ", color=:blue)
      p2 = plot_tensor(Δ_rand; title="Scrambled Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

**Warning:** More than 1,000 viewable entries, plotting only the largest points.  
@ DletoPlotsExt

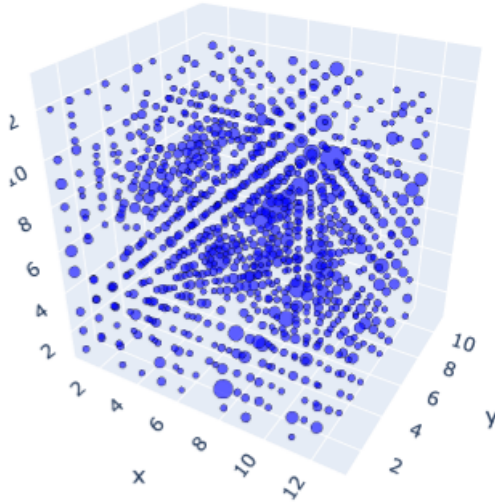
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257

**Warning:** More than 1,000 viewable entries, plotting only the largest points.  
@ DletoPlotsExt

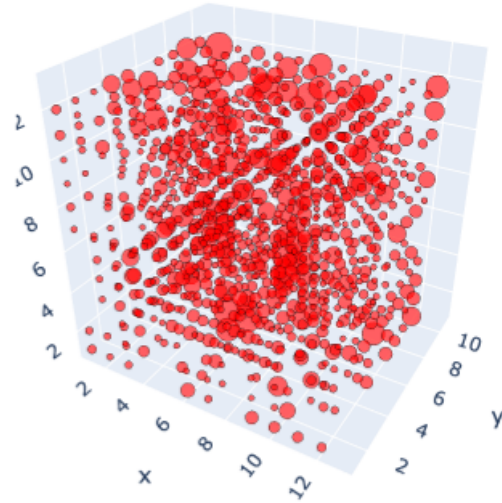
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257

[17]:

Scrambled Control  $\Gamma$



Scrambled Experiment  $\Delta$



This at least provides visual assurance that we have hidden the block structure in our experiment tensor. Let us now try to use chiseling to recover it:

```
[18]: Γ_strat, X2s = stratify(Γ_rand; tol=tol)    # specify tol to match noise level
      @assert isapprox(Γ_rand*X2s, Γ_strat)
      Δ_strat, Y2s = stratify(Δ_rand; tol=tol)
      @assert isapprox(Δ_rand*Y2s, Δ_strat)
```

[ Info: Found 2 derivations for stratification.

[ Info: Found 10 derivations for stratification.

The volume of the dots are proportional to the scalars they represent. In particular, tiny values are omitted entirely from the plots. Inspecting the actual data provides more detail. Up a small tolerance we see that we have recovered the degeneracy embedded in the experimental tensor.

```
[19]: p1 = plot_tensor(Γ_strat, tol; title="Stratified Control Γ", color=:blue)
      p2 = plot_tensor(Δ_strat, tol; title="Stratified Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

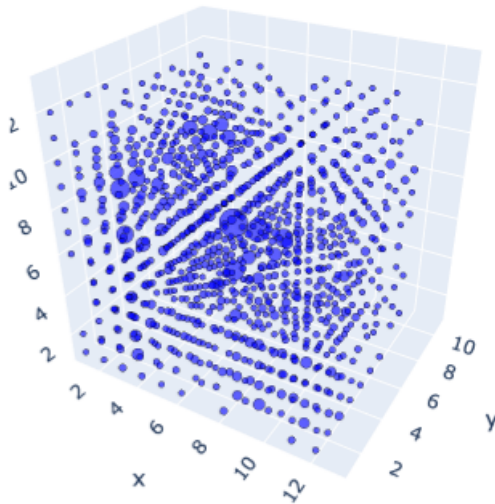
**Warning:** More than 1,000 viewable entries, plotting only the largest points.

@ DletoPlotsExt

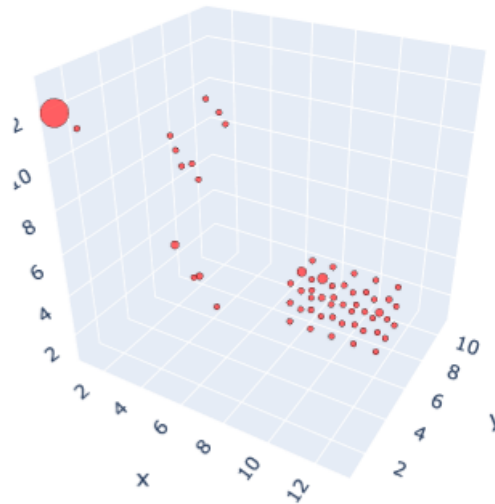
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257

[19]:

Stratified Control  $\Gamma$



Stratified Experiment  $\Delta$



Once more we have recovered some structure. The order of our blocks has changed (and it may be helpful to interact with 3D plot to understand the structure).

We again invite you to revisit the experiment and add noise, add more blocks, and change dimensions.

### 2.0.1 Carry out the trash

If you are ready for the next experiment you might get better viewing results if you release some more trash.

```
[20]: # release memory of tensors
       $\Gamma$  =nothing;  $\Delta$ =nothing
       $\Gamma$ _rand=nothing;  $\Delta$ _rand=nothing
       $\Gamma$ _strat=nothing;  $\Delta$ _strat=nothing
      X1s=nothing; Y1s=nothing
      X2s=nothing; Y2s=nothing

      # release memory plots. Note, in some Jupyter environments, the 3D plots will
      ↪ persist as an image but no longer be interactive.
```

```
p1=nothing
p2=nothing
GC.gc()
```

## 2.1 3 Stratification

In linear algebra there are many uses for diagonals, block diagonals, and triangular forms of matrices. Moving from matrices to tensors with more than two axes, we encounter something new and unrelated to matrices: curves and surfaces.

In keeping with previous demonstrations, we carry out a controlled experiment.

```
[21]: ds = (30,30,30)
      Γ = randn(Float64, ds...) # a control tensor

      db = 0.0
      tol = 1e-4

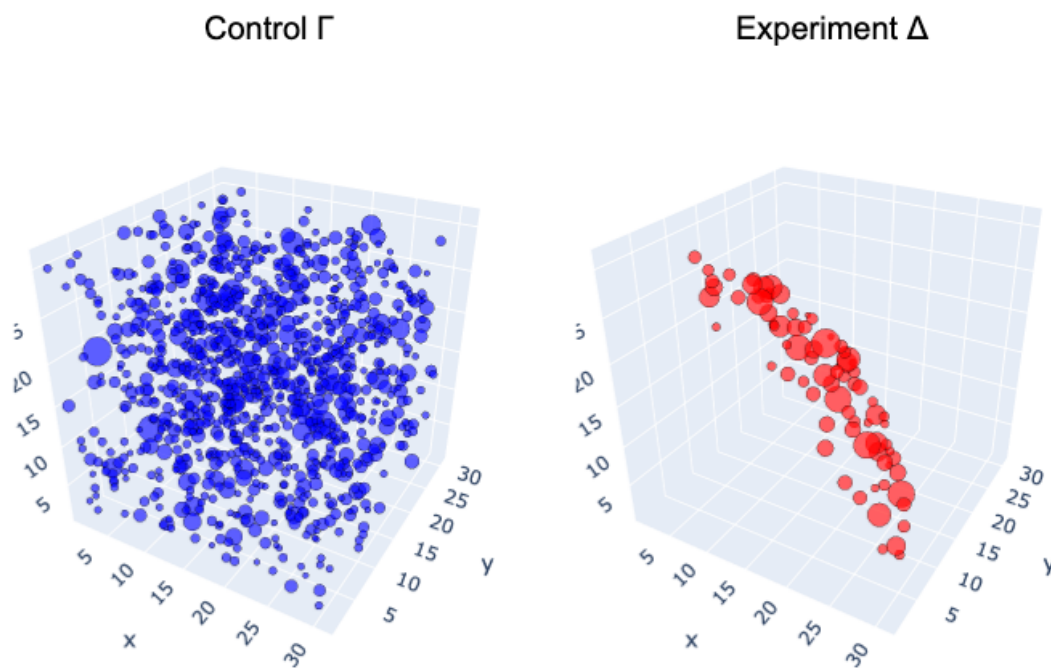
      # An experiment tensor with a hidden surface.
      # - Choose f(x,y,z) to define the surface
      f(x,y,z) = x^2 + y^2 + z^2
      # - Populate Δ near the surface defined by f
      Δ = zeros(Float64, ds...);
      for i in 1:ds[1], j in 1:ds[2], k in 1:ds[3]
          if abs( f(i,j,k) - 30^2 ) < 1 + 2*randn()
              Δ[i,j,k] = randn()
          end
      end

      # to avoid long wait times raise the threshold for plotting
      p1 = plot_tensor(Γ; title="Control Γ", color=:blue)
      p2 = plot_tensor(Δ; title="Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

**Warning:** More than 1,000 viewable entries, plotting only the largest points.  
 © DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257

[21]:



Dleto supplies a range of convenient functions to generate tensors of this sort. It also provides configurable noise and distance functions.

```
[22]: Γ_rand, X1s = randomize_tensor(Γ)
      @assert isapprox(Γ * X1s, Γ_rand)
      Δ_rand, Y1s = randomize_tensor(Δ)
      @assert isapprox(Δ * Y1s, Δ_rand)
```

The randomized versions are by now visually familiar:

```
[23]: p1 = plot_tensor(Γ_rand; title="Scrambled Control Γ", color=:blue)
      p2 = plot_tensor(Δ_rand; title="Scrambled Experiment Δ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

```
Warning: More than 1,000 viewable
entries, plotting only the largest points.
@ DletoPlotsExt
```

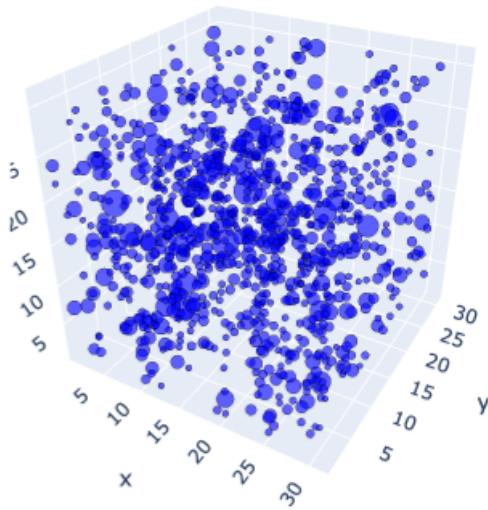
```
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257
```

```
Warning: More than 1,000 viewable
entries, plotting only the largest points.
@ DletoPlotsExt
```

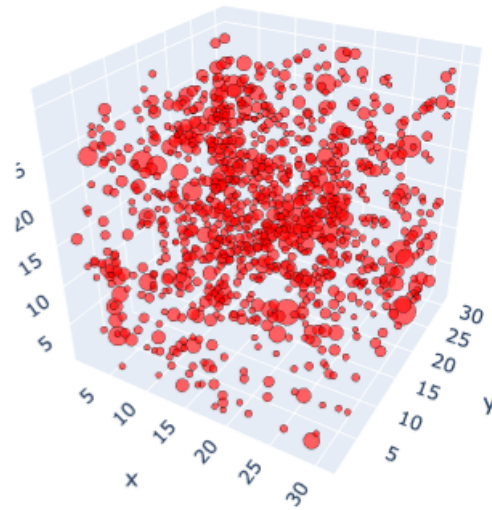
```
~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257
```

[23]:

Scrambled Control  $\Gamma$



Scrambled Experiment  $\Delta$



```
[24]: @time  $\Gamma$ _strat, X2s = stratify( $\Gamma$ _rand; tol=1e-2)    # specify tol to match noise
      ↪ level
      @assert isapprox( $\Gamma$ _rand*X2s,  $\Gamma$ _strat)
```

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[ Info: Found 2 derivations for stratification.

7.423270 seconds (31.00 M allocations: 8.433 GiB, 9.49% gc time, 2.00% compilation time)

Since we are trying to stratify larger surfaces, it makes sense (as we suggested earlier) to first remove any degeneracies:

```
[25]:  $\Delta$ _nondeg, Y2s = Dleto.nondeg( $\Delta$ _rand, mode=:trunc);
      @assert isapprox( $\Delta$ _rand*Y2s,  $\Delta$ _nondeg)
      @time  $\Delta$ _strat, Z1s = stratify( $\Delta$ _nondeg; tol=1e-1)
      @assert isapprox( $\Delta$ _nondeg*Z1s,  $\Delta$ _strat)
```

Using SVDSolver...

Converting LinearMap to Matrix for SVD...

[ Info: Found 10 derivations for stratification.

3.721225 seconds (21.58 M allocations: 5.282 GiB, 18.79% gc time)

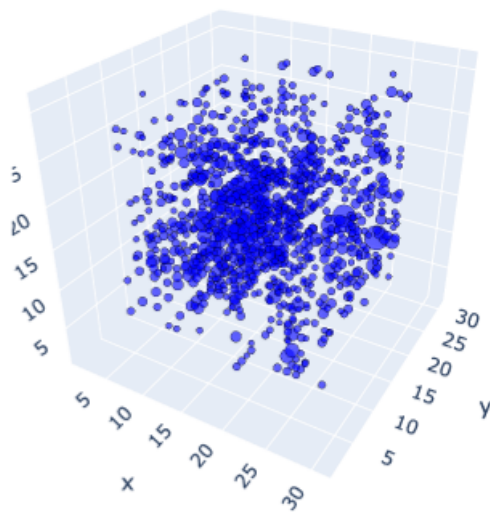
```
[26]: p1 = plot_tensor( $\Gamma$ _strat; title="Stratified Control  $\Gamma$ ", color=:blue)
      p2 = plot_tensor( $\Delta$ _strat; title="Stratified Experiment  $\Delta$ ", color=:red)
      plot(p1, p2, layout=layout, size=pic_size)
```

**Warning:** More than 1,000 viewable entries, plotting only the largest points.  
@ DletoPlotsExt

~/CODE/OpenDleto/ext/DletoPlotsExt.jl:257

[26]:

Stratified Control  $\Gamma$



Stratified Experiment  $\Delta$

